Single Responsibility Principle (SRP)

We designed each class in the system to only have one single, well-defined responsibility. The ApiIngestionService, for example, is responsible only for ingesting and normalising external transport data, while the ApiHealthService focuses solely on monitoring API availability and latency. RouteVerificationService is set up to cross-check route data across multiple sources, and ReliabilityService is responsible only for computing reliability scores. In the database model, the entities such as Route, Station, and ServiceStatus each represent one distinct domain concept. Reducing any overlap to the minimum.

Open–Closed Principle (OCP)

The entities and services are structured so that new functionality can be added later without modifying existing code. For example, new transport data providers can be added by implementing the TransportApiClient interface without changing the ingestion or verification services. Extra reliability factors can be added by extending the logic in ReliabilityService rather than rewriting it. This allows our system to scale while maintaining stability. Hence, the open-closed principle.

Liskov Substitution Principle (LSP)

All implementations of the TransportApiClient interface can be substituted without affecting system behaviour. Services such as ApiIngestionService depend on the interface rather than on specific external API implementations. This allows different transport providers to be swapped without impacting the ingestion process.

Interface Segregation Principle (ISP)

Interfaces and repositories only access the functionality required by the services that use them. For example, ReliabilityService depends only on ReliabilityRepository and VerificationRepository and does not depend on, say, the ingestion-related repositories. Then you can also take the TransportApiClient interface, which contains only methods relevant to retrieving transport data. This setup avoids unnecessary dependencies and keeps the services loosely coupled.

Dependency Inversion Principle (DIP)

All the high-level services depend on the abstractions rather than the actual concrete implementations. Core logic components only interact with the repositories or the API clients through interfaces instead of directly accessing databases or external APIs. For instance, RouteVerificationService uses the VerificationRepository rather than directly interacting with SQL code. This separation improves testability, maintainability, and flexibility of the system architecture.